Midterm Writeup: Matrix-Free Spectral Elements in 3D on Elliptic Problem

Fall 2026

In this project, we implemented a parallel, matrix-free spectral element solver for a 3D elliptic problem on the unit cube $\Omega = [0,1]^3$. The model problem is

$$-\nabla^2 u + u = f \ \text{ in } \Omega, \qquad u = 0 \ \text{ on } \partial\Omega. \tag{1}$$

To test for convergence, we preselected the manufactured solution

$$u(x,y,z) = \sin(4\pi x)\sin(2\pi y)\sin(3\pi z),$$

which vanishes on all faces of the cube. Since $-\nabla^2 u = (4^2+2^2+3^2)\pi^2 u = 29\pi^2 u$, this leads to the forcing function

$$f(x,y,z) = (29\pi^2 + 1)\, u(x,y,z).$$

Multiplying Equation (1) by a test function v with $v = 0$ on $\partial\Omega$, integrating over the domain, and integrating by parts (the boundary term vanishes because $v = 0$ on $\partial\Omega$) gives the weak form

$$\int_\Omega \nabla u \cdot \nabla v \, dV + \int_\Omega u\, v \, dV = \int_\Omega f\, v \, dV.$$

With the spectral element method (SEM), we partition the domain into E non-overlapping elements and require the weak form to hold over the whole domain, assembled as a sum of element contributions with C^0 continuity across element faces. The integrals are approximated with Gauss-Lobatto-Legendre (GLL) quadrature. The GLL nodes on the reference interval also serve as interpolation nodes for the Lagrange polynomials ℓ_i that form the basis of the spectral expansion on each element:

$$u(x,y,z) = \sum_{k=0}^{N_z}\sum_{j=0}^{N_y}\sum_{i=0}^{N_x} u_{ijk}\, \ell_i(x)\ell_j(y)\ell_k(z).$$

The test function is expanded in the same basis (a Galerkin method). Using Kronecker product notation, the element-local system is

$$
\begin{aligned}
&\mathbf{v}_e^T \left(\hat{M}_z \otimes \hat{M}_y \otimes \hat{K}_x\right) \mathbf{u}_e + \quad \mathbf{v}_e^T \left(\hat{M}_z \otimes \hat{K}_y \otimes \hat{M}_x\right) \mathbf{u}_e + \\
+&\mathbf{v}_e^T \left(\hat{K}_z \otimes \hat{M}_y \otimes \hat{M}_x\right) \mathbf{u}_e + \quad \mathbf{v}_e^T \left(\hat{M}_z \otimes \hat{M}_y \otimes \hat{M}_x\right) \mathbf{u}_e = \\
=&\mathbf{v}_e^T \left(\hat{M}_z \otimes \hat{M}_y \otimes \hat{M}_x\right) \mathbf{f}_e.
\end{aligned}
$$

where $\hat{K}$ and $\hat{M}$ are the 1D stiffness and mass matrices, respectively, including the element-size scaling (approximately $2/h$ for $\hat{K}$ and $h/2$ for $\hat{M}$ in each direction on affine elements). Under GLL quadrature the mass matrices are diagonal. The global operator is $A = Q^T A_L Q$, where A_L is block-diagonal over elements and Q is the scatter operator that copies global node values to element-local storage; Q^T is the corresponding gather (sum). Homogeneous Dirichlet nodes are handled by masking.

Applying each Kronecker product as a sequence of small 1D tensor contractions (sum-factorization) reduces the operation count per element from $O(N^6)$ for a dense element matrix to $O(N^4)$, and the storage from $O(N^6)$ to $O(N^3)$.

For parallelization, the unit-cube domain was distributed using Recursive Coordinate Bisection (RCB) to achieve load balancing. The gather/scatter operations, which are addition/copy routines used to communicate and update node values shared across processors, were handled by gslib. Because the system is symmetric positive definite, a straightforward parallel conjugate gradient (CG) solver was implemented This algorithm is particularly well-suited for SEM because it only requires the *action* of a matrix on a vector, and it only requires a couple global reductions per iteration.

Our implementation achieved exponential (spectral) convergence in a p-type convergence study. In an h-type convergence study with polynomial degree $N = 3$ and $N = 5$, we observed fourth- and sixth-order convergence, respectively, in the L_2 norm, consistent with the expected $O(h^{N+1})$ rate in L^2. On a strong-scaling study (43986977 d.o.f.s), $> 80\%$ parallel efficiency was maintained up to about 64 MPI ranks, after which communication dominated. A similar trend was seen in a weak-scaling study (~704969 d.o.f.s). As the number of ranks grew, the fraction of time spent in the Kronecker product evaluations, the most computationally expensive part of the algorithm, fell below the time spent in `Allreduce` and became comparable to that of the gather-scatter routine. This indicates that at high rank counts the collective communication in CG, rather than the local tensor-product kernels, limits scalability.

Parallel Matrix-Free Spectral Elements for Solving 3D Elliptic Problem

CS 598 Midterm

[1]Liam Pohlmann, [2]Alex Shaffer

November 2026

[1]Department of Nuclear, Plasma, and Radiological Engineering

[2]Department of Computer Science

Table of Contents

1. Project Description
2. Numerical Model
   - Weak Form
   - Spectral Element Method
3. Implementation
   - Kronecker Product Evaluation
   - Recursive Coordinate Bisection
   - Gather/Scatter Operator
   - Iterative Solver
4. Results
5. References

1. Project Description

Toy Problem

For this implementation, we look to solve the following toy problem:

Find $\tilde{u} \in \mathcal{R}^3$ s.t.

$$-\nabla^2 \tilde{u} + \tilde{u} = f(x, y, z), \quad \tilde{u} \in \Omega, \quad u \in \partial\Omega = 0. \tag{1}$$

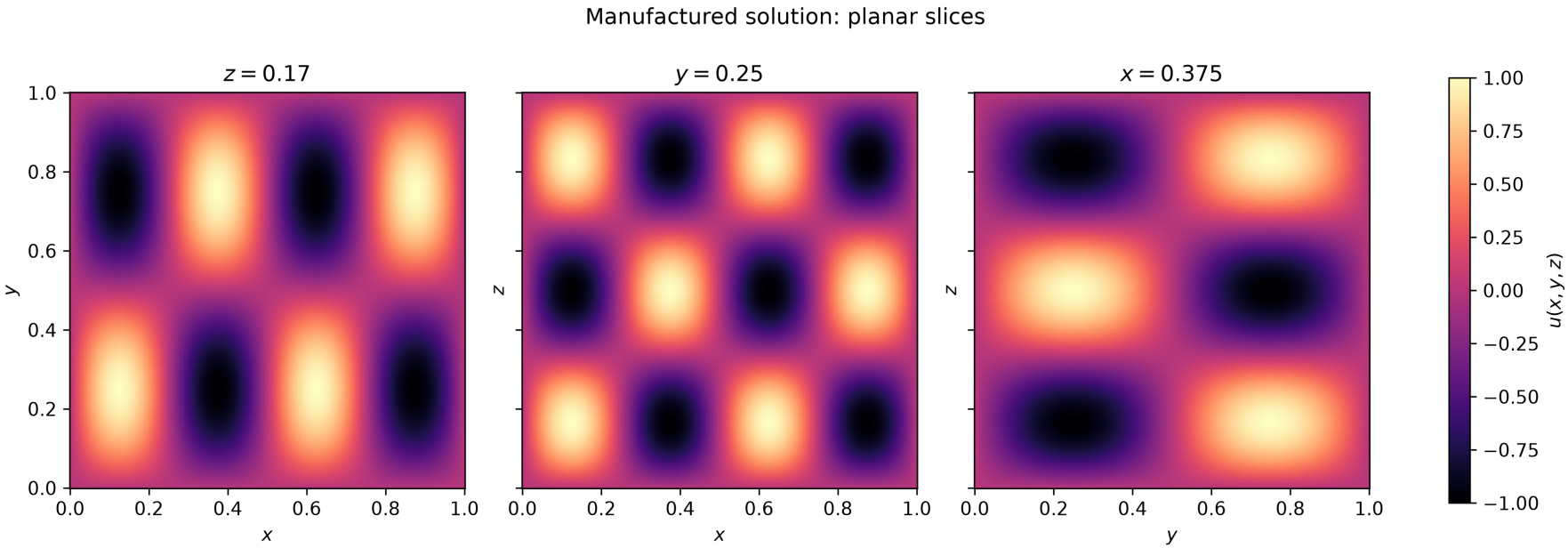
for $x, y, z \in [0, 1]^3$. We additionally choose the following manufactured solution:

$$\tilde{u} = \sin(4\pi x) \sin(2\pi y) \sin(3\pi z), \quad k \in \mathcal{R}^+ \tag{2}$$

This sets the forcing function to be:

$$\begin{aligned} f(x, y, z) &= \left(29\pi^2 + 1\right) \sin(k\pi x) \sin(k\pi y) \sin(k\pi z) \\ &= \left(29\pi^2 + 1\right) \tilde{u}(x, y, z) \end{aligned} \tag{3}$$

MMS Plot


Manufactured solution: planar slices
$z = 0.17$
$y = 0.25$
$x = 0.375$
y
z
z
x
x
y
$u(x, y, z)$
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

2. Numerical Model

2. Numerical Model

2.1. Weak Form

Weak Form of PDE

To develop the weak form, we first approximate the exact solution with a numerical solution, $\tilde{u} \approx u$, then multiply both sides of Equation (1) by a test function, v, and integrate over the domain:

$$-\int v\nabla^2 u \,\mathrm{d}V + \int uv \,\mathrm{d}V = \int vf \,\mathrm{d}V \tag{4}$$

Using vector identities and integration by parts leaves:

$$\int \nabla u \cdot \nabla v \,\mathrm{d}V - \oint v\nabla u \cdot \hat{n} \,\mathrm{d}S + \int uv \,\mathrm{d}V = \int fv \,\mathrm{d}V. \tag{5}$$

Galerkin Approximation

We first choose the approximation u to be a sum of Lagrange basis functions:

$$u(x,y,z) = \sum_{k=0}^{N_z} \sum_{j=0}^{N_y} \sum_{i=0}^{N_x} u_{ijk} \ell_i(x) \ell_j(y) \ell_k(z) \tag{6}$$

for N_x, N_y, and N_z elements in the x, y, z directions, respectively. For a Galerkin projection, we choose:

$$v(x,y,z) = \sum_{n=0}^{N_z} \sum_{m=0}^{N_y} \sum_{l=0}^{N_x} v_{lmn} \ell_l(x) \ell_m(y) \ell_n(z). \tag{7}$$

Using Equation (6) and Equation (7) in Equation (8) gives a set of $(N_x+1)(N_y+1)(N_z+1)$ equations.

Galerkin Approximation

Because of our choice for the test function in Equation (7), v also satisfies the homogeneous Dirichlet boundary conditions, leaving our weak form simply:

$$\underbrace{\int \nabla u \cdot \nabla v \, \mathrm{d}V + \int uv \, \mathrm{d}V}_{\mathcal{A}(u,v)} = \underbrace{\int fv \, \mathrm{d}V}_{\mathcal{F}(v)} . \tag{8}$$

Affine Transformation

As a quick aside, we define the affine transformation, which is valid for $x_i \in [0, L_i]$:

$$x_i(r_i) = \frac{L_i}{2}(1 + r_i)\,. \tag{9}$$

This mapping transforms $x, y, z \in [0, L] \to \mu, \eta, \xi \in [-1, 1]$, simplifying the integrals in the weak form.

Kronecker Product Form

Applying the weak form to a set of E discretized hexahedral elements, each with a tensor product of internal nodes, gives $\forall e \in \{1, 2, \ldots, E\}$:

$$\begin{aligned}
&\mathbf{v}_e^T \left(\hat{M}_z \otimes \hat{M}_y \otimes \hat{K}_x \right) \mathbf{u}_e + \\
&\quad \mathbf{v}_e^T \left(\hat{M}_z \otimes \hat{K}_y \otimes \hat{M}_x \right) \mathbf{u}_e + \mathbf{v}_e^T \left(\hat{K}_z \otimes \hat{M}_y \otimes \hat{M}_x \right) \mathbf{u}_e + \\
&\quad \mathbf{v}_e^T \left(\hat{M}_z \otimes \hat{M}_y \otimes \hat{M}_x \right) \mathbf{u}_e = \mathbf{v}_e^T \left(\hat{M}_z \otimes \hat{M}_y \otimes \hat{M}_x \right) \mathbf{f}_e. \quad (10)
\end{aligned}$$

$\hat{K}$ and $\hat{M}$ are the element *stiffness* and *mass* matrices, respectively. Note that the forcing function, $f(x, y, z)$, has been transformed using Equation (9) and expanded using Lagrange polynomials.

Kronecker Product Form

The stiffness and mass matrices have the following definitions:

$$\begin{cases} \left(\hat{M}_x\right)_{il} = \frac{L_x}{2} \int_{-1}^{1} \ell_i(\mu)\ell_l(\mu)\,\mathrm{d}\mu \\ \left(\hat{M}_y\right)_{jm} = \frac{L_y}{2} \int_{-1}^{1} \ell_j(\eta)\ell_m(\eta)\,\mathrm{d}\eta \\ \left(\hat{M}_z\right)_{kn} = \frac{L_z}{2} \int_{-1}^{1} \ell_j(\xi)\ell_n(\xi)\,\mathrm{d}\xi \end{cases} \tag{11}$$

$$\begin{cases} \left(\hat{K}_x\right)_{ij} = \frac{2}{L_x} \int_{-1}^{1} \frac{\mathrm{d}\ell_i}{\mathrm{d}\mu} \frac{\mathrm{d}\ell_l}{\mathrm{d}\mu}\,\mathrm{d}\mu \\ \left(\hat{K}_y\right)_{jm} = \frac{2}{L_y} \int_{-1}^{1} \frac{\mathrm{d}\ell_j}{\mathrm{d}\eta} \frac{\mathrm{d}\ell_m}{\mathrm{d}\eta}\,\mathrm{d}\eta \\ \left(\hat{K}_z\right)_{kn} = \frac{2}{L_z} \int_{-1}^{1} \frac{\mathrm{d}\ell_k}{\mathrm{d}\xi} \frac{\mathrm{d}\ell_n}{\mathrm{d}\xi}\,\mathrm{d}\xi. \end{cases} \tag{12}$$

2. Numerical Model

2.2. Spectral Element Method

Spectral Element Method

The defining feature of the spectral element method (SEM) is the selection of internal element nodes corresponding to the Gauss-Lobatto-Legendre (GLL) rule. From [1]:

$$
\begin{aligned}
\int_{-1}^{1} u(\xi)\,\mathrm{d}\xi \quad &\approx \sum_{k=0}^{N} \rho_k\, u\left(\xi_k\right) \\
\xi_k \qquad &= \begin{cases} -1, & k = 0 \\ \text{zeros of } L'_N(\xi), & 1 \le k \le N-1 \\ 1, & k = N \end{cases} \\
\rho_k \qquad &= \frac{2}{N\left(N+1\right)} \frac{1}{\left[L_N\left(\xi_k\right)\right]^2}
\end{aligned}
\tag{13}
$$

The above rule is exact for polynomials degree up to $2N-1$.

Spectral Element Method

Using the points in Equation (13), which we approximate the integrals for the stiffness and mass matrices. Using *generic* indices i, j, we have:

$$\left(\hat{M}_n\right)_{ij} \approx \frac{L_n}{2} \sum_{k=0}^{N} \rho_k \ell_i(\mu_k) \ell_j(\mu_k) = \begin{cases} \rho_i, & i = j \\ 0, & i \neq j \end{cases} \tag{14a}$$

$$\left(\hat{K}_n\right)_{ij} \approx \frac{2}{L_n} \sum_{k=0}^{N} \rho_k \frac{\mathrm{d}\ell_i}{\mathrm{d}\mu}(\mu_k) \frac{\mathrm{d}\ell_j}{\mathrm{d}\mu}(\mu_k) \tag{14b}$$

Notice that because the GLL quadrature points are also the nodes for the Lagrange interpolants, the mass matrix is *diagonal*. The stiffness matrix, however, is full.

3. Implementation

3. Implementation

3.1. Kronecker Product Evaluation

Kronecker Product

For three square matrices of the same size, $A, B, C \in \mathbb{R}^{(N+1)\times(N+1)}$, the Kronecker product can be expanded using $(N+1) \times (N+1)$ identity matrices:

$$A \otimes B \otimes C = (A \otimes I \otimes I)(I \otimes B \otimes I)(I \otimes I \otimes C). \tag{15}$$

We will be interested in finding the product, $(A \otimes I \otimes I)(I \otimes B \otimes I)(I \otimes I \otimes C)\mathbf{u}$, where $\mathbf{u}$ is a length $(N+1)^3$ vector. Let:

$$\mathbf{w} = (I \otimes I \otimes C)\mathbf{u} \tag{16a}$$

$$\mathbf{v} = (I \otimes B \otimes I)\mathbf{w} \tag{16b}$$

$$\mathbf{y} = (A \otimes I \otimes I)\mathbf{v} \tag{16c}$$

Kronecker Product

Define the flat indexing:

$$\hat{\imath}(i,j,k) = i + j(N+1) + k(N+1)^2. \tag{17}$$

Then we have the following definitions:

$$w_{\hat{\imath}} = w_{ijk} = \sum_{p=0}^{N} C_{ip} u_{pjk} \tag{18a}$$

$$v_{\hat{\imath}} = v_{ijk} = \sum_{p=0}^{N} B_{jp} w_{ipk} \tag{18b}$$

$$y_{\hat{\imath}} = y_{ijk} = \sum_{p=0}^{N} A_{kp} v_{ijp}. \tag{18c}$$

Note that in doing this, we reduce the matrix-vector product from $\mathcal{O}\left((N+1)^6\right)$ to $\mathcal{O}\left((N+1)^4\right)$, recovering two orders of magnitude.

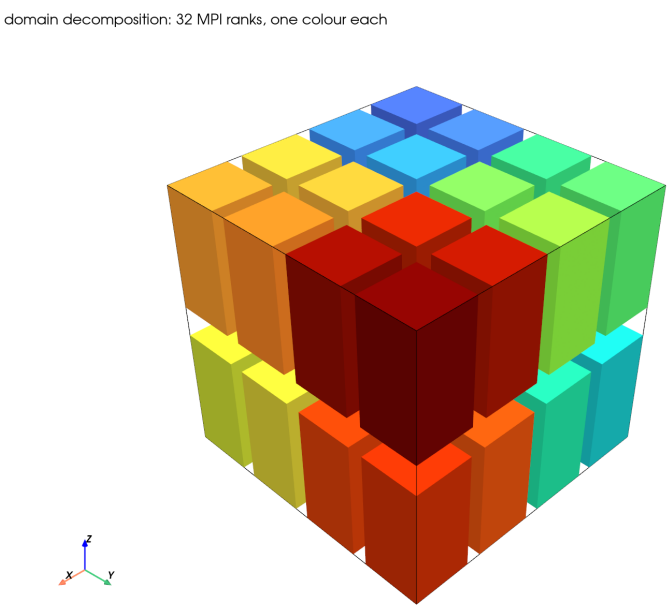
3. Implementation

3.2. Recursive Coordinate Bisection

Recursive Coordinate Bisection

Ranks own a subset of the elements in the mesh and the DOFs associated with those elements.

Recursive Coordinate Bisection (RCB) is used to achieve load balancing for the parallelization.



Recursive Coordinate Bisection

Algorithm 1 RCB of element set E onto ranks $r_0, \ldots, r_0 + P - 1$

1: **procedure** $\mathrm{RCB}(E,\ r_0,\ P)$
2: **if** $P = 1$ **then**
3: assign every $e \in E$ to rank r_0
4: **return**
5: **end if**
6: $a \leftarrow$ longest axis of the bounding box of the centroids of E
7: sort E by centroid coordinate along a ▷ ties: element ID
8: $P_L \leftarrow \lfloor P/2 \rfloor, \quad P_R \leftarrow P - P_L$
9: $n_L \leftarrow \lfloor |E|\, P_L/P \rfloor$ ▷ elements $\propto$ ranks
10: $\mathrm{RCB}(E[1 : n_L],\ r_0,\ P_L)$
11: $\mathrm{RCB}(E[n_L + 1 : |E|],\ r_0 + P_L,\ P_R)$
12: **end procedure**

3. Implementation

3.3. Gather/Scatter Operator

Gather-Scatter Operator

As discussed in [1], the *scatter* operator copies node values on element boundaries, while the *gather* operator sums these values together. This allows for a single sharing of values after each iteration and updating of all nodes.

$$\begin{aligned} \mathbf{u}_L &= Q\mathbf{u} && \text{scatter} \\ \mathbf{v} &= Q^T\mathbf{u}_L && \text{gather} \end{aligned} \tag{19}$$

This was handled using `gslib` (`https://github.com/nek5000/gslib`).

3. Implementation

3.4. Iterative Solver

Conjugate Gradient

Conjugate Gradient (CG) solves the linear system $A\mathbf{x} = \mathbf{b}$ for a symmetric positive definite matrix A iteratively.

It is very well suited for SEM because it:

- Only requires the action of the matrix A on a vector.
- Only requires a couple global reductions per iteration.

Conjugate Gradient

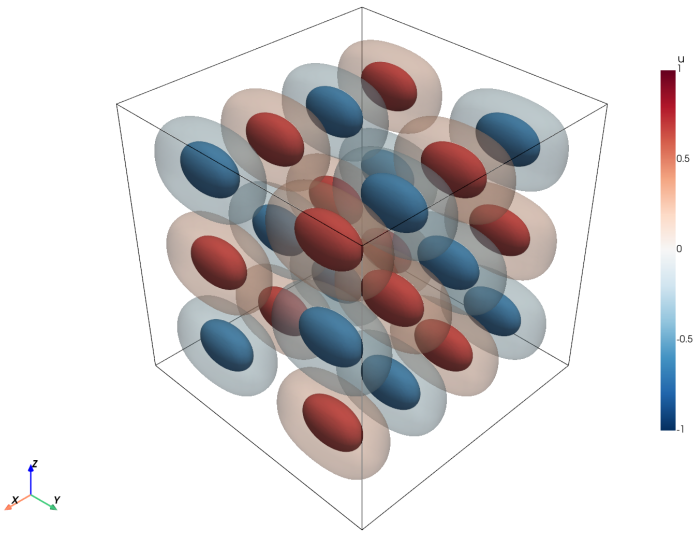
Algorithm 2 Unpreconditioned CG for $Au = b$

1: $u \leftarrow 0, \quad r \leftarrow b, \quad p \leftarrow r$
2: $\rho \leftarrow r^T r$
3: **while** $\sqrt{\rho} \geq \varepsilon \|b\|$ **do**
4: $\quad w \leftarrow Ap$ ▷ Kronecker products + gather-scatter
5: $\quad \alpha \leftarrow \rho/(p^T w)$ ▷ `MPI_Allreduce`
6: $\quad u \leftarrow u + \alpha p$
7: $\quad r \leftarrow r - \alpha w$
8: $\quad \rho_{\text{new}} \leftarrow r^T r$ ▷ `MPI_Allreduce`
9: $\quad p \leftarrow r + (\rho_{\text{new}}/\rho)\, p$
10: $\quad \rho \leftarrow \rho_{\text{new}}$
11: **end while**

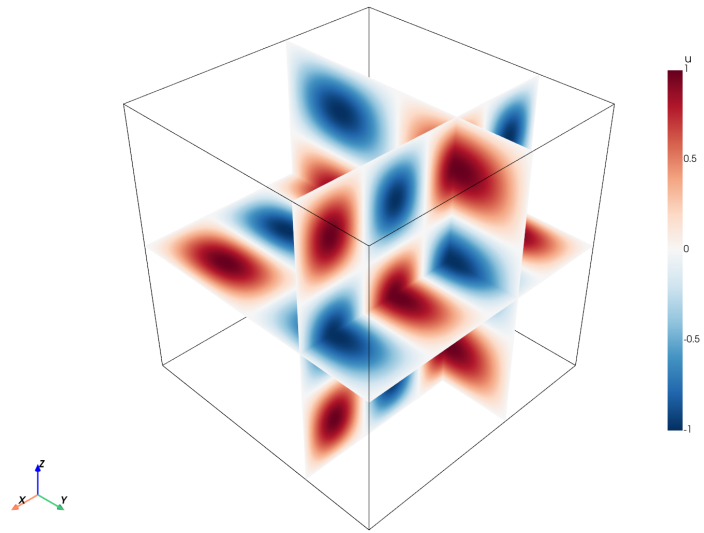
4. Results

Solution

u: isosurfaces at 0.3 and 0.75 of the peak value, both signs

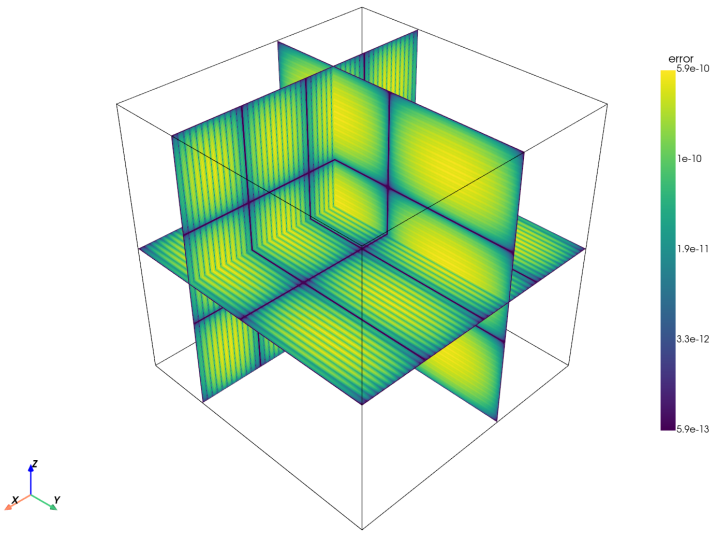


u: slices through (0.379, 0.729, 0.5)

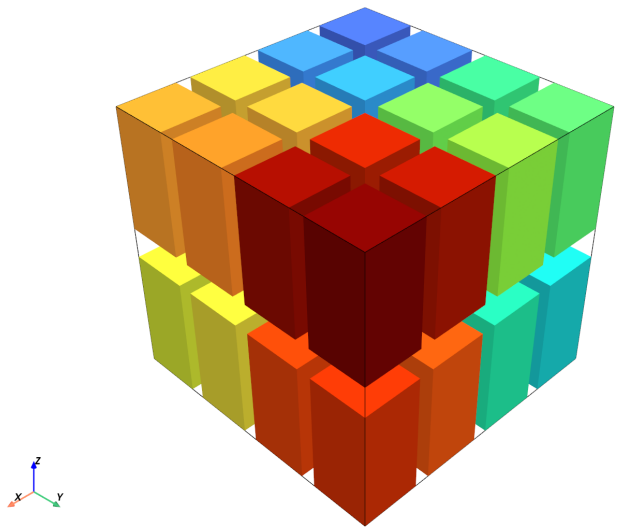


Solution Error and Ranks

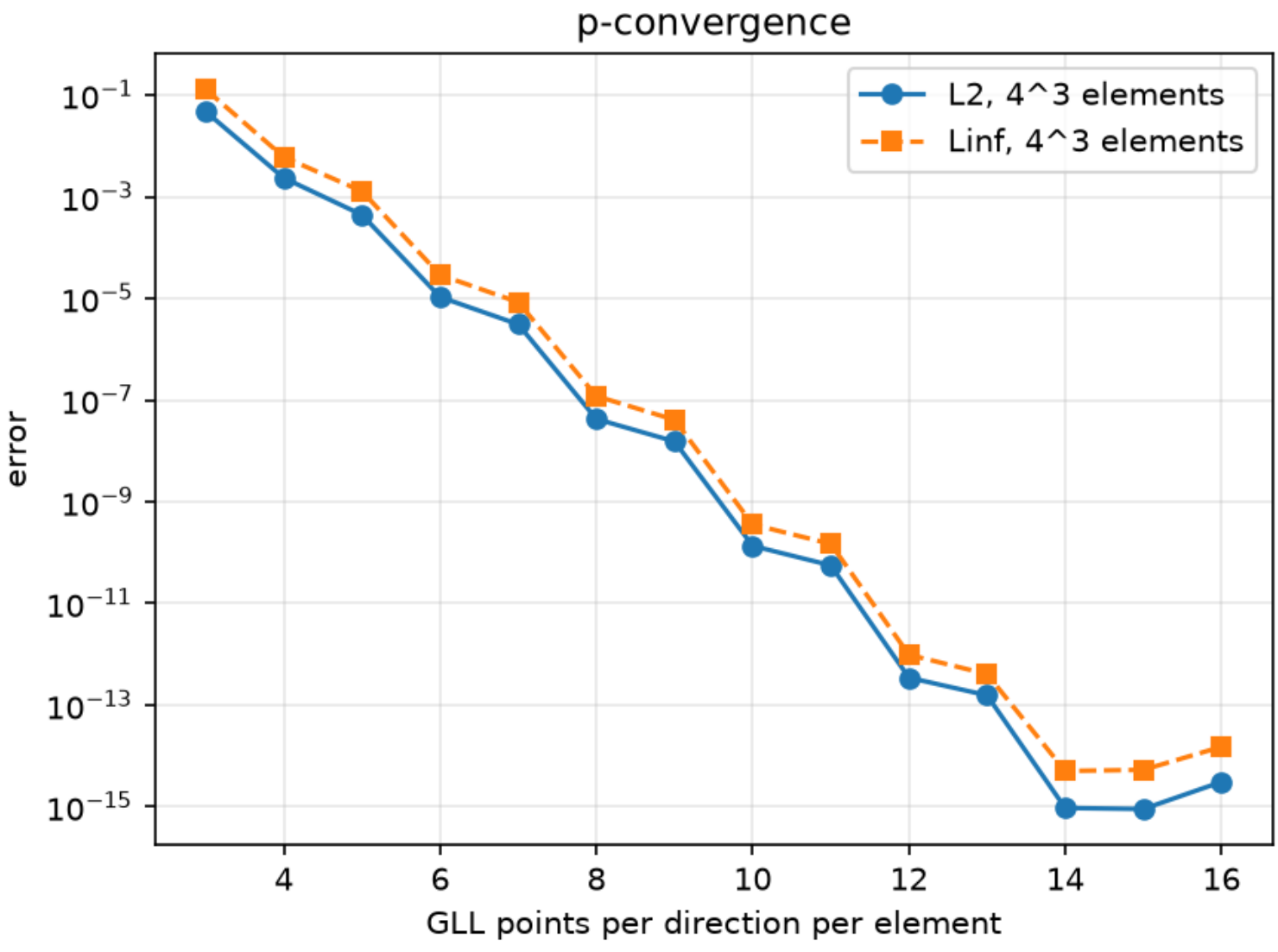
error magnitude, u - u_exact (max 5.86e-10)



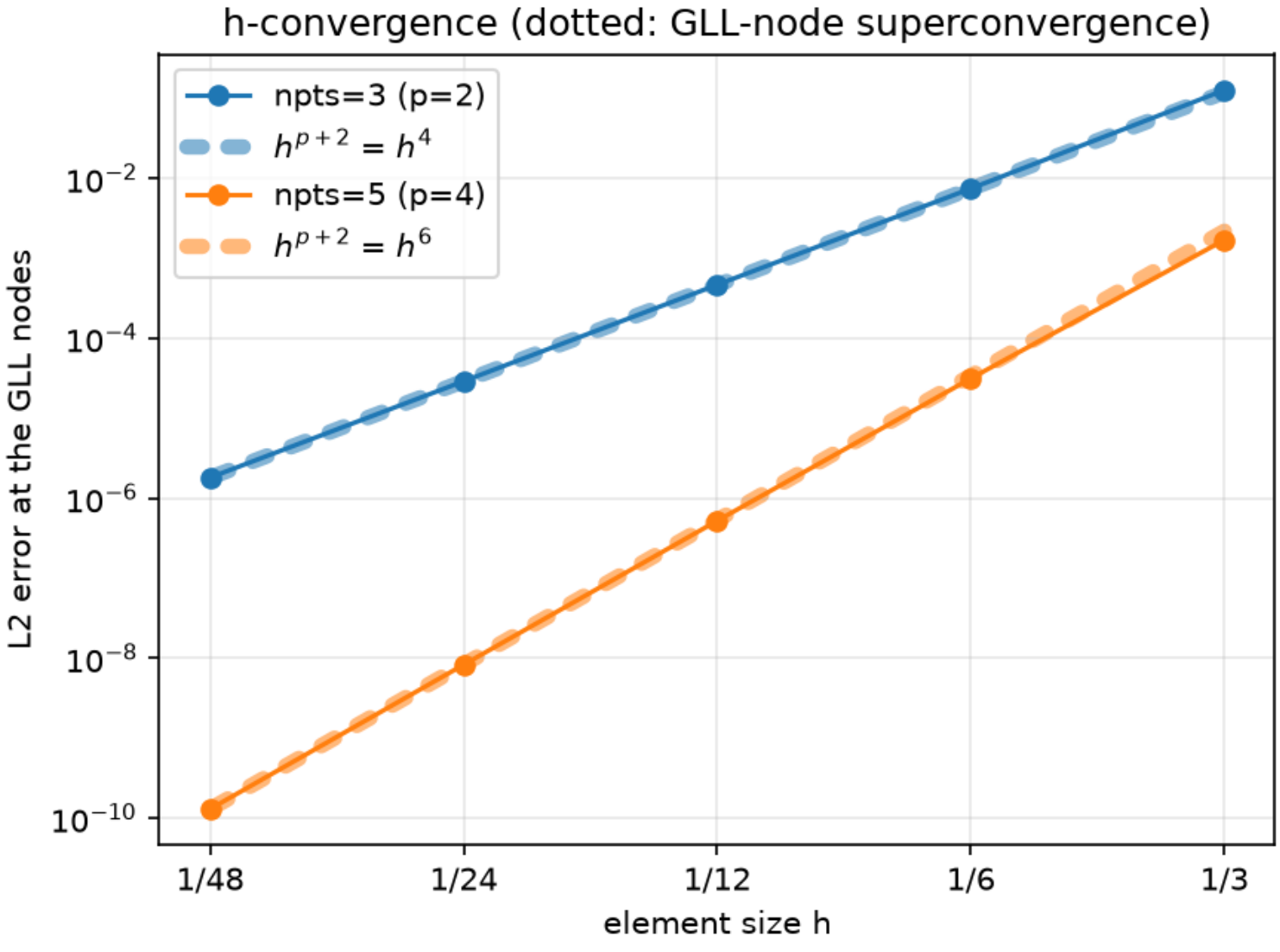
domain decomposition: 32 MPI ranks, one colour each



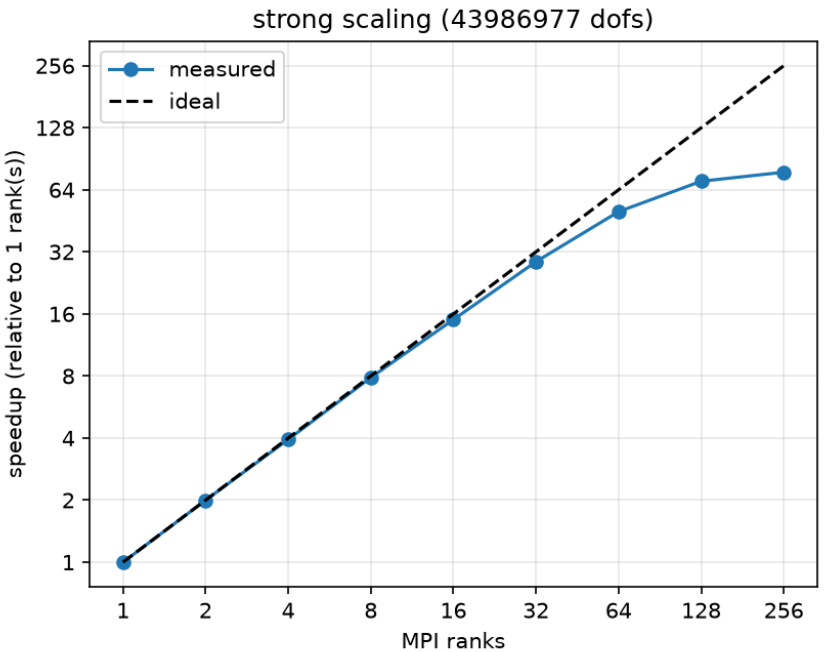
p-type Convergence

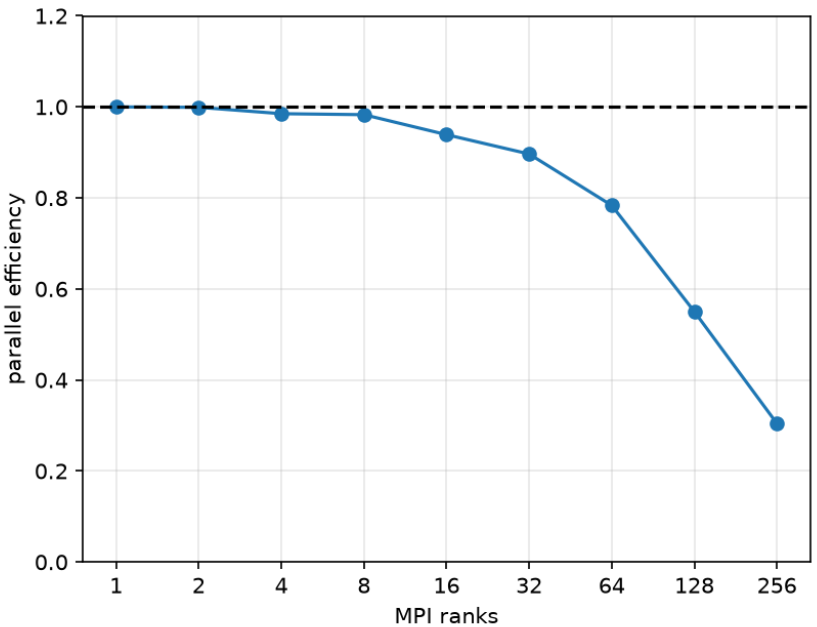


h-type Convergence

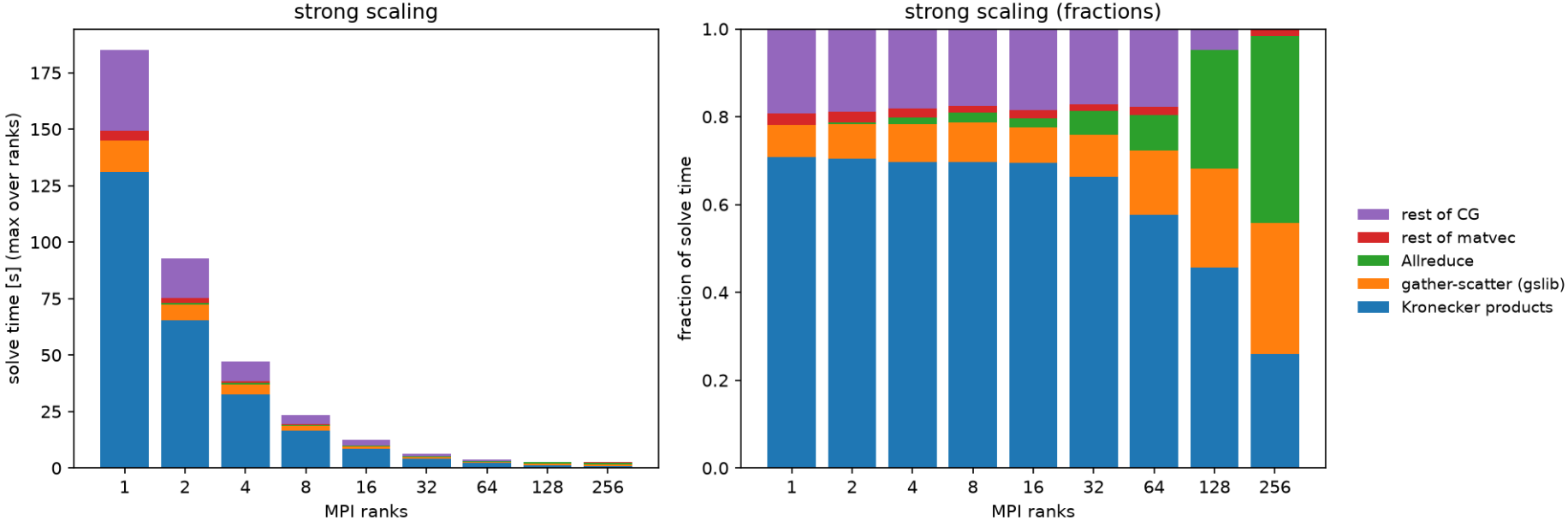


Strong Scaling

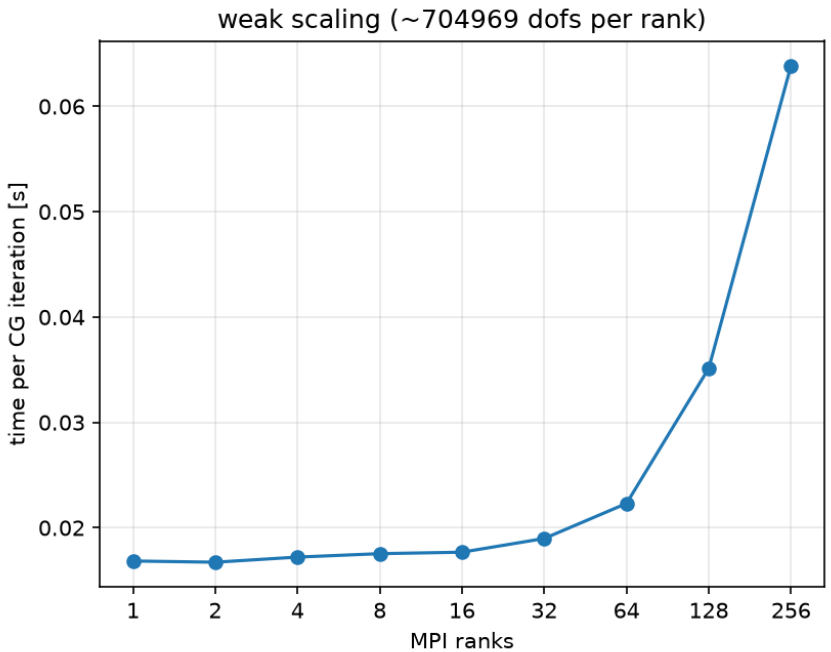


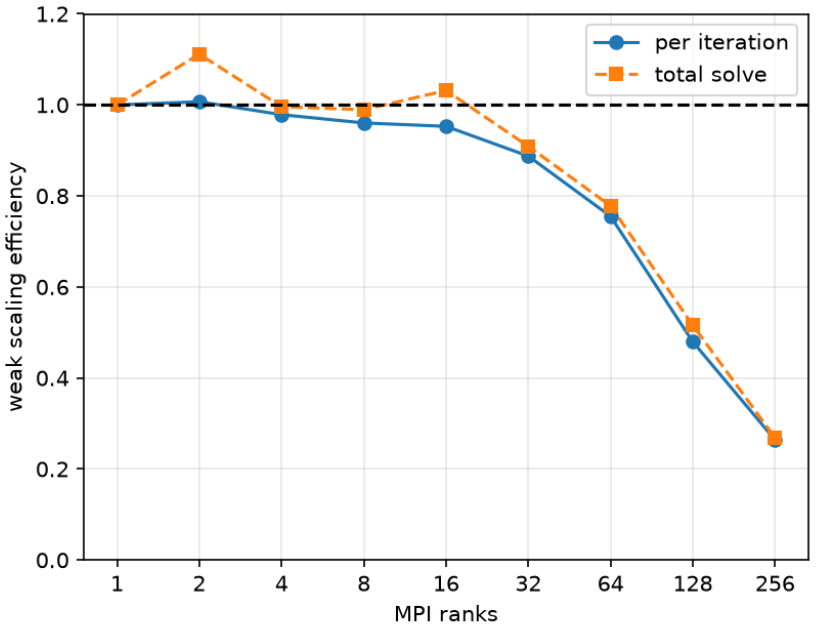


Strong Scaling

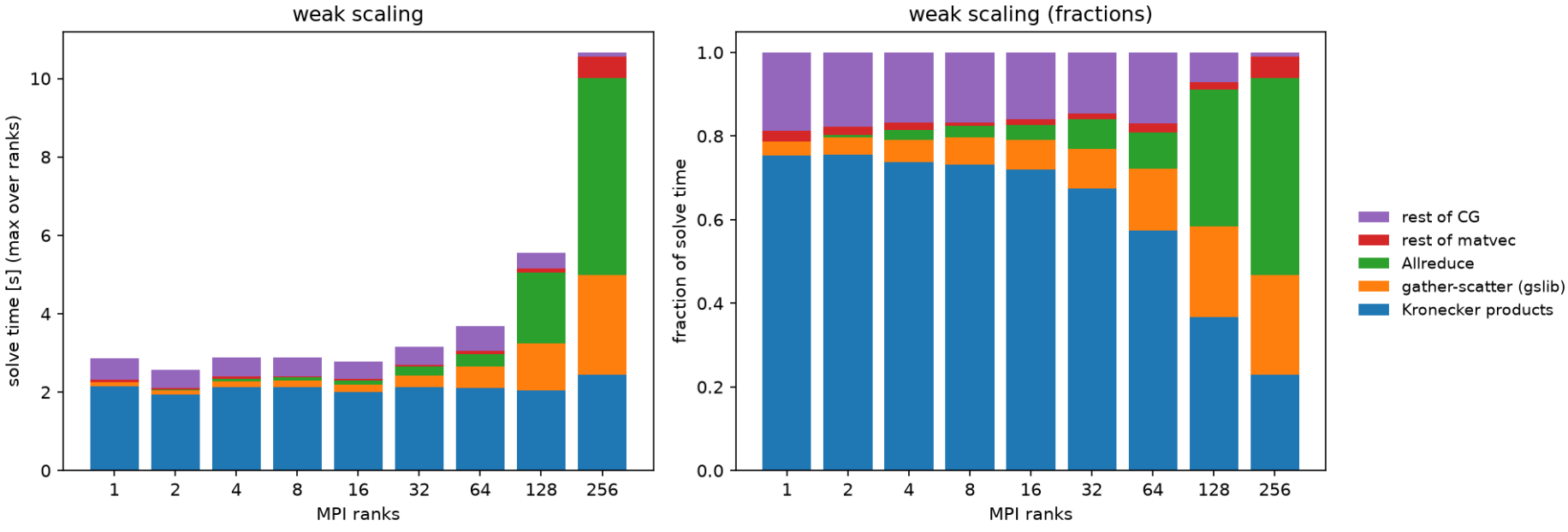


Weak Scaling





Weak Scaling



5. References

References (i)

[1] Michel Deville, P. F. Fischer, and E. H. Mund, eds. *High-Order Methods for Incompressible Fluid Flow*. Cambridge Monographs on Applied and Computational Mathematics 9. Cambridge, UK New York: Cambridge University Press, 2002. 499 pp. ISBN: 978-0-521-45309-7 978-0-511-03946-1 978-0-511-05284-2 978-0-511-09197-1 978-0-511-54679-2.